ING1 — Développement Web

Encadré par les enseignants de CY Tech

Année universitaire 2025 / 2026

Rapport de projet

Maison Intelligente

Plateforme web de gestion d'objets connectés

Réalisé par :

Ziyad Haddadi — Khayem Ben Ghorbel — Rindra Rakotonirina — Anastasia Tsybulskaya — Valérie Roux

Stack : Django REST Framework + JWT · React + Vite · SQLite

Sommaire

I. Introduction et contexte

- 1. Présentation du projet
- 2. Choix du thème
- 3. Outils et technologies retenus

II. Répartition des tâches

- 1. Organisation du groupe
- 2. Tableau de répartition
- 3. Diagramme de Gantt

III. Étapes réalisées

- 1. Phase 1 — Conception et modélisation (UML, BDD)
- 2. Phase 2 — Backend Django + API REST + JWT
- 3. Phase 3 — Frontend React + intégration
- 4. Phase 4 — Modules avancés et finitions

IV. Présentation des modules

- Module Information (visiteur)
- Module Visualisation (utilisateur simple)
- Module Gestion (utilisateur complexe)
- Pouvoirs étendus de l'administrateur

V. Problèmes rencontrés et solutions

VI. Conclusion et perspectives

I. Introduction et contexte

1. Présentation du projet

Dans le cadre de notre première année du cycle d'ingénieur (ING1) à CY Tech, nous avons réalisé un projet de développement web s'inscrivant dans le contexte de l'Internet des Objets (IoT) et de la domotique. Notre plateforme, baptisée « Maison Intelligente », permet de centraliser la gestion d'objets connectés (thermostats, caméras, lave-linge, volets, etc.) et de services domotiques au sein d'une habitation.

L'objectif est de proposer une plateforme numérique qui regroupe divers services et fonctionnalités adaptés aux besoins de différents types d'utilisateurs (visiteurs, résidents, gestionnaires, administrateur). La plateforme reçoit des données issues des objets connectés, permet de les visualiser, de les paramétrer et d'en gérer la maintenance.

Le projet a été conçu pour répondre à plusieurs exigences pédagogiques : architecture modulaire en quatre niveaux d'accès, base de données réelle, authentification sécurisée, recherche avec filtres, système de progression par points et niveaux, ainsi que conception responsive et accessible.

2. Choix du thème

Parmi les trois thèmes proposés (ville/village intelligent, bâtiment intelligent, établissement intelligent), nous avons retenu le bâtiment intelligent au travers d'une maison connectée. Ce choix se justifie par plusieurs raisons :

- Le périmètre fonctionnel est clair et délimité : un foyer comporte un nombre fini de pièces et d'objets, ce qui facilite la modélisation.
- Les cas d'usage sont concrets et intuitifs : tout étudiant comprend immédiatement ce qu'est un thermostat de salon ou une caméra d'entrée.
- Les rôles utilisateurs (parent, enfant, visiteur) correspondent à des familles de droits clairement distinguables.
- La domotique est un domaine en forte croissance et représente un sujet de soutenance attractif.

3. Outils et technologies retenus

- Backend : Django 5 et Django REST Framework (DRF) pour la création de l'API REST. Django offre un ORM puissant, un système d'authentification robuste, et un panneau d'administration intégré. DRF permet d'exposer les modèles sous forme de serializers REST en quelques lignes.
- Authentification : SimpleJWT pour générer des jetons JWT (JSON Web Token), standard moderne pour les API. Le jeton d'accès dure 12 heures, le jeton de rafraîchissement 7 jours.
- Base de données : SQLite, choisie pour sa simplicité (aucun serveur à installer, fichier unique) et son intégration native avec Django via les migrations.
- Frontend : React 18 avec Vite comme outil de build, axios pour les requêtes HTTP, et React Router pour la navigation côté client.
- CORS : django-cors-headers pour autoriser le frontend (port 5173) à dialoguer avec le backend (port 8000) en développement.
- Versionnement : Git avec dépôt distant, commits réguliers, structure de dossiers claire (backend / frontend séparés).

II. Répartition des tâches

1. Organisation du groupe

Notre groupe est composé de cinq étudiants. Avant de commencer le développement, nous avons tenu une réunion de cadrage durant laquelle nous avons défini les modules et les responsabilités de chacun. Nous avons opté pour une méthode agile inspirée de Scrum, avec des points d'avancement réguliers, malgré le peu de temps pour réaliser le projet et un outil collaboratif Notion pour suivre les tâches en cours, à faire et terminées.

Le dépôt Git nous a permis de travailler simultanément en limitant les conflits, chacun sur ses fichiers principaux puis avec des branches pour les fonctionnalités plus larges. Discord nous a servi de canal de communication asynchrone pour partager des captures, des extraits de code et des liens utiles.

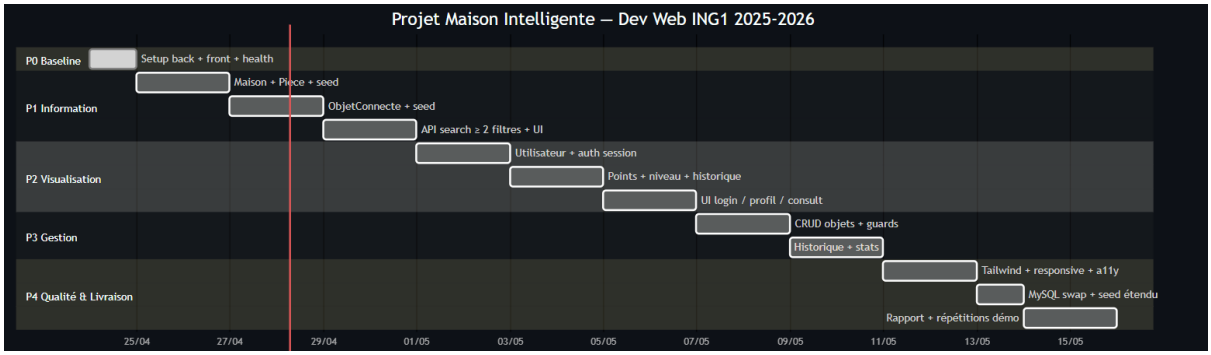
2. Tableau de répartition

Étudiant	Tâches principales	Part de travail
Ziyad	Modélisation UML, modèles Django (User, Room, Device), API REST des objets connectés, recherche avec filtres, gestion des pièces et catégories, Authentification JWT.	35 %
Khayem	Modélisation UML, système de points et niveaux, profil utilisateur (photo, mot de passe), gestion des actions et de l'historique.	20 %
Rindra	Modélisation UML, Frontend React, composants Navbar et ProtectedRoute, design responsive et accessibilité	15 %
Anastasia	Modélisation UML, Frontend React , design responsive et accessibilité, page d'administration des demandes de suppression.	15 %
Valérie	Modélisation UML, Frontend React, page Statistiques avec exports CSV, rédaction du rapport.	15 %

Certaines tâches transverses ont été réalisées collectivement : choix du thème, modélisation initiale, conception du diagramme UML, charte graphique (couleurs principales : bleu nuit #26215C et indigo #4F46E5), structuration des dossiers, et tests de bout en bout.

3. Diagramme de Gantt

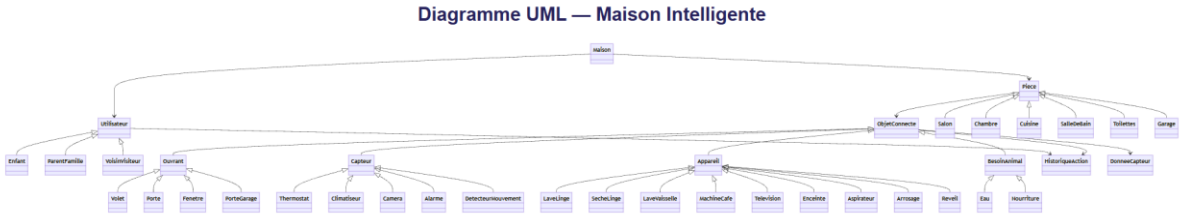
Le projet a été découpé en cinq phases successives, chacune d'une durée approximative de 1 semaine :



III. Étapes réalisées

1. Phase 1 — Conception et modélisation

Avant d'écrire la moindre ligne de code, nous avons modélisé le domaine métier sous forme d'un diagramme UML de classes. Cette étape a permis de fixer le vocabulaire et les relations avant de figer le schéma de base de données.



Les classes principales identifiées sont :

- User : utilisateur de la plateforme, avec attributs publics (pseudo, âge, genre, date de naissance, rôle, photo) et privés (nom, prénom, mot de passe). Hérite d'AbstractUser de Django pour bénéficier nativement du système d'authentification.
- Room : pièce de la maison (salon, chambre, cuisine, garage, jardin, etc.).
- Category : catégorie d'objet (Sécurité, Confort, Électroménager, Divertissement, Extérieur).

- Device : objet connecté avec ses attributs métier (type, marque, batterie, valeur courante, valeur cible, horaires de fonctionnement, dernière maintenance).
- Service : outil ou service proposé par la plateforme (suivi consommation, alarme intrusion, mode nuit, etc.).
- Action : événement d'historique (qui a fait quoi, quand).
- Stat : mesure de consommation énergétique d'un objet à une date donnée.
- DeletionRequest : demande de suppression d'un objet (workflow utilisateur → admin).

Les relations 1-N (un User a plusieurs Action, une Room contient plusieurs Device, etc.) ont été codées en clés étrangères Django (ForeignKey).

2. Phase 2 — Backend Django + API REST + JWT

Nous avons créé un projet Django nommé smart_home et une application api regroupant tous les modèles. Les principales étapes :

1. Définition des modèles dans api/models.py avec les contraintes (max_length, choices, valeurs par défaut).
2. Création du système de migrations (makemigrations puis migrate) pour générer automatiquement le schéma SQLite.
3. Écriture des serializers DRF dans api/serializers.py — un serializer par modèle, avec gestion des champs read-only et de la liaison photo (URL absolue construite via le contexte de la requête).
4. Création des vues : ViewSets pour les ressources (Device, Service, Room, Category, DeletionRequest) et fonctions @api_view pour les endpoints spécifiques (login, profile, set-level, exports CSV, maintenance, history, stats).
5. Configuration de SimpleJWT avec un sérialiseur personnalisé qui ajoute 0,25 point à chaque connexion et génère une entrée Action.
6. Surcharge de la méthode retrieve du DeviceViewSet pour ajouter 0,5 point à l'utilisateur connecté à chaque consultation d'objet.
7. Implémentation du workflow de demande de suppression : un utilisateur avancé crée une DeletionRequest, l'administrateur l'approuve (l'objet est supprimé) ou la refuse depuis une page dédiée.
8. Commande de management seed.py pour peupler la base avec des données d'exemple (5 catégories, 7 pièces, 4 utilisateurs + admin, 15 objets, 10 services, 7 jours de stats par objet).

3. Phase 3 — Frontend React + intégration

Le frontend a été initialisé avec Vite. Nous avons retenu une approche sans framework UI lourd : juste React, react-router-dom et axios. Le CSS est natif et écrit en mobile-first avec des media queries pour les écrans larges.

La structure du dossier src/ est organisée en :

- api.js : configuration centrale d'axios (intercepteur ajoutant le JWT à chaque requête) et helpers (saveAuth, refreshUser, isLoggedIn, isAdvanced, isAdmin).
- components/ : composants réutilisables (Navbar avec menus déroulants, ProtectedRoute pour gérer les niveaux d'accès).
- pages/ : une page React par route (Home, Login, Register, Dashboard, Devices, DeviceDetail, Profile, etc.).

- utils.js : fonctions transverses (helper dash() pour afficher un tiret au lieu de zéros vides).
- styles.css : feuille de style globale avec design responsive, contraste WCAG, focus visible.

L'intégration s'est faite progressivement : nous avons d'abord fait fonctionner l'inscription et la connexion bout en bout, puis ajouté les pages une par une en testant à chaque étape la communication React ↔ Django.

4. Phase 4 — Modules avancés et finitions

Cette dernière phase a consisté à compléter les fonctionnalités manquantes et à corriger les défauts d'ergonomie remontés lors des tests croisés :

- Page de maintenance avec détection automatique des objets à réviser (batterie inférieure à 20 % ou dernière maintenance datant de plus de six mois) et bouton Marquer comme réparé qui recharge la batterie et met la date de maintenance à aujourd'hui.
- Page d'administration des demandes de suppression : liste des demandes en attente avec boutons Supprimer et Refuser. Une fois traitée, la demande disparaît automatiquement des deux côtés (utilisateur et admin).
- Choix manuel du niveau via la page /level : l'utilisateur peut sélectionner n'importe quel niveau pour lequel il a accumulé assez de points.
- Upload de photo de profil avec invalidation du cache navigateur (paramètre de version dans l'URL).
- Changement de mot de passe avec vérification de l'ancien.
- Exports CSV des objets et de la consommation énergétique.
- Rafraîchissement automatique des données utilisateur à chaque changement de route (compteurs de connexions et d'actions toujours à jour).
- Refonte de la barre de navigation en trois menus déroulants (Explorer, Mon espace, Gestion) afin de la rendre moins chargée.
- Restriction de l'accès aux détails des objets pour le visiteur non connecté (cartes affichées en grisé avec invitation à se connecter).

IV. Présentation des modules

Conformément au cahier des charges, la plateforme est divisée en plusieurs modules accessibles selon le niveau de l'utilisateur. Chaque module correspond à un ensemble cohérent de fonctionnalités.

Module Information (visiteur)

Le visiteur, c'est-à-dire toute personne arrivant sur la plateforme sans être connectée, a accès à un « free tour » : une page d'accueil qui présente le projet, ainsi qu'un catalogue en lecture seule des objets connectés et des services proposés.

Le moteur de recherche embarque cinq filtres simultanés : recherche textuelle (nom, description, marque), type d'objet, pièce, catégorie et état (actif / inactif). Les filtres sont indépendants et combinables, ce qui dépasse l'exigence de deux filtres minimum imposée par le cahier des charges.

Le visiteur peut accéder à un bouton d'inscription depuis n'importe quelle page. En revanche, il ne peut pas cliquer sur les fiches d'objets ou de services pour en voir le détail : ces routes sont protégées et redirigent vers la page de connexion.

Module Visualisation (utilisateur simple)

Une fois inscrit et connecté, l'utilisateur accède à un tableau de bord personnel. Le formulaire d'inscription récupère toutes les informations exigées par le PDF (pseudo, email, mot de passe, prénom, nom, âge, genre, date de naissance, rôle dans la maison).

Le profil est divisé en deux parties :

- Partie publique consultable par tous les membres : pseudo, âge, genre, date de naissance, rôle, photo.
- Partie privée consultable uniquement par l'utilisateur lui-même et l'administrateur : nom, prénom, changement de mot de passe.

Le tableau de bord affiche en temps réel le niveau actuel, le nombre de points cumulés, le nombre de connexions effectuées, le nombre d'actions, ainsi qu'un historique des événements récents (déduplication des entrées par identifiant pour éviter tout doublon).

Le système de points fonctionne ainsi : 0,25 point pour chaque connexion, 0,5 point pour chaque consultation d'un objet ou d'un service. Quatre niveaux sont définis : Débutant (0 pt), Intermédiaire (5 pts), Avancé (10 pts) et Expert (15 pts). Conformément au demande du projet, l'utilisateur peut choisir manuellement son niveau parmi ceux qu'il a débloqués via la page dédiée /level.

Module Gestion (utilisateur complexe)

Au niveau Avancé (10 points), l'utilisateur débloque le module Gestion. Trois nouvelles fonctionnalités principales apparaissent dans la barre de navigation :

- Création et modification d'objets connectés avec tous leurs paramètres : nom, type, marque, description, pièce, catégorie, batterie, valeur courante et cible, horaires de fonctionnement, date de dernière maintenance, état.
- Activation/désactivation immédiate d'un objet via un bouton de bascule (toggle), avec mise à jour temps réel et trace dans l'historique.
- Demande de suppression d'un objet : conformément au PDF qui demande de « solliciter à l'administrateur la suppression », l'utilisateur ne supprime pas directement

mais envoie une requête, motif compris. Il peut suivre l'état de ses demandes sur la page /my-requests.

Le module Gestion donne aussi accès à trois pages d'analyse :

- Maintenance : liste filtrée des objets dont la batterie est faible ou dont la dernière révision est trop ancienne, avec bouton de réparation.
- Historique : journal global des actions effectuées sur les objets, tous utilisateurs confondus, avec filtre textuel.
- Statistiques : vue d'ensemble (nombre d'objets, objets actifs, consommation totale, répartition par type et par pièce) et boutons d'export CSV (objets et consommation).

Pouvoirs étendus de l'administrateur

L'administrateur cumule tous les droits des autres niveaux et bénéficie en plus de pouvoirs spécifiques :

- Sur la page de détail d'un objet, le bouton « Demander suppression » est remplacé par « Supprimer » qui exécute la suppression immédiatement.
- Une page /admin-requests, accessible depuis le menu Gestion, liste toutes les demandes de suppression en attente de tous les utilisateurs. Chaque demande peut être approuvée (l'objet est supprimé) ou refusée. Dans les deux cas, la demande disparaît ensuite des listes.
- Le panneau d'administration Django natif (/admin) reste accessible et permet de gérer finement utilisateurs, points, niveaux, catégories, pièces, services, et de consulter l'historique complet.

V. Problèmes rencontrés et solutions

Conflit de migrations Django sur le User personnalisé

Lors de la première installation, nous avons rencontré l'erreur InconsistentMigrationHistory : les migrations de l'app admin avaient été appliquées avant celles de notre app api, ce qui empêchait la création du modèle User personnalisé. La cause : nous avions oublié de créer la migration initiale de notre app avant le premier migrate.

Solution : suppression complète du fichier db.sqlite3 et des migrations existantes, puis exécution dans le bon ordre : python manage.py makemigrations api suivi de python manage.py migrate. La règle pour la suite : toujours déclarer les modèles personnalisés et créer leurs migrations avant le tout premier migrate.

Actions doublées dans l'historique

Au début, chaque consultation d'un objet créait deux entrées dans l'historique au lieu d'une. Après investigation, nous avons identifié que React StrictMode (activé par défaut en mode développement) exécute volontairement chaque useEffect deux fois pour aider à détecter les effets de bord.

Solution : suppression de StrictMode dans main.jsx pour éviter le doublement des appels API en développement, et ajout d'une déduplication par identifiant côté frontend en sécurité supplémentaire dans le composant Dashboard.

Photo de profil non rafraîchie après upload

Après avoir uploadé une nouvelle photo, le navigateur continuait d'afficher l'ancienne image, même après rechargement de la page. Cause : le navigateur met en cache l'image associée à l'URL, et comme le serveur renvoyait la même URL, aucune nouvelle requête n'était déclenchée.

Solution : ajout d'un paramètre de version dans l'URL (img?v=timestamp). À chaque upload réussi, un nouveau timestamp est généré, ce qui force le navigateur à considérer l'URL comme nouvelle et à recharger l'image.

Cascade de suppression sur les demandes traitées

Lors de l'approbation d'une demande de suppression, l'administrateur recevait un message d'erreur alors que l'objet était bien supprimé. Cause : le modèle DeletionRequest avait une clé étrangère device avec on_delete=CASCADE. En supprimant l'objet, Django supprimait aussi la demande, ce qui empêchait ensuite de mettre à jour son statut à « approuvée ».

Solution : passage de la clé étrangère en on_delete=SET_NULL avec null=True, puis dans la vue, on détache d'abord la FK (dr.device = None), on enregistre le statut, puis on supprime le device. Ainsi la demande est conservée et tracée correctement.

Barre de navigation surchargée

La première version de la barre listait tous les liens à plat : Accueil, Objets, Services, Tableau de bord, Profil, Niveau, Maintenance, Historique, Statistiques, Mes demandes, Connexion, Inscription, Déconnexion. Sur mobile, la navigation occupait deux lignes et nuisait à l'ergonomie.

Solution : refonte en trois menus déroulants thématiques (Explorer, Mon espace, Gestion) qui regroupent les liens par usage. La barre est maintenant compacte sur tous les supports, et le menu se ferme automatiquement lorsque l'utilisateur clique en dehors ou change de page.

Données de profil obsolètes après navigation

Le compteur d'actions dans le tableau de bord ne se mettait pas à jour entre les visites de pages : il fallait se reconnecter ou rafraîchir manuellement la page pour voir l'incrément.

Solution : ajout d'un useEffect dans la Navbar qui appelle refreshUser() à chaque changement de location.pathname. Cela rafraîchit silencieusement le profil dans le localStorage, et la valeur affichée dans le composant suivant reflète l'état réel du backend.

Sécurité : suppression directe par les utilisateurs avancés

Initialement, n'importe quel utilisateur de niveau Avancé pouvait supprimer un objet via DELETE /api/devices/<id>/. Cela contredisait le PDF qui exige que l'utilisateur complexe « sollicite à l'administrateur la suppression ».

Solution : surcharge de la méthode destroy du DeviceViewSet pour rejeter avec un code HTTP 403 toute tentative de suppression émanant d'un utilisateur non administrateur. Le frontend a été modifié en conséquence : bouton « Demander suppression » pour les utilisateurs avancés (qui ouvre un formulaire de motif), bouton « Supprimer » direct pour l'administrateur.

VI. Conclusion et perspectives

Ce projet de plateforme de maison intelligente nous a permis de mettre en pratique l'ensemble des compétences acquises en développement web tout au long de l'année. Au-delà de l'aspect purement technique, il nous a obligés à structurer une réflexion sur le cycle de vie complet d'une application : modélisation, conception d'API, sécurité, ergonomie, accessibilité et tests.

Sur le plan fonctionnel, nous avons couvert l'intégralité des exigences du cahier des charges :

- Quatre niveaux d'utilisateurs avec droits différenciés (visiteur, simple, complexe, administrateur).
- Modules Information, Visualisation et Gestion accessibles selon le niveau, plus pouvoirs étendus pour l'administrateur.
- Inscription complète avec validation email simulée par token, vérification du membre par l'administrateur, validation login/mot de passe.
- Profil public et privé avec toutes les informations demandées (pseudo, âge, genre, date de naissance, rôle, photo, nom, prénom).
- Recherche multi-filtres sur les objets (cinq filtres simultanés) et sur les services (trois filtres).
- Système de points (0,25 par connexion, 0,5 par consultation) avec calcul automatique du niveau et choix manuel.
- Gestion complète des objets : ajout, modification, demande de suppression, activation/désactivation, configuration des horaires.
- Maintenance : détection automatique des objets à réviser, bouton de réparation.
- Statistiques globales et exports CSV (objets et consommation).
- Historique global et personnel des actions.
- Conception responsive mobile-first et conformité WCAG (skip link, balises sémantiques, contrastes, focus visible).
- Dépôt Git avec commits réguliers et structure de dossiers claire.

Sur le plan technique, nous avons appris à utiliser Django REST Framework de manière idiomatique (ViewSets, serializers, permissions, parsers multipart), à intégrer JWT pour l'authentification stateless, à structurer un projet React de taille moyenne, et à faire dialoguer les deux briques via CORS.

Perspectives

Plusieurs axes d'amélioration ont été identifiés pour aller plus loin :

- Mise en place de WebSockets (Django Channels) pour pousser en temps réel les changements d'état des objets vers le frontend (par exemple, voir s'allumer une alarme déclenchée par un autre utilisateur sans rafraîchir la page).
- Ajout d'un système de notifications dans l'application (cloche en haut à droite) pour informer un utilisateur que sa demande de suppression a été traitée, ou qu'un objet nécessite une maintenance.
- Intégration d'un véritable envoi d'emails (SMTP ou service comme SendGrid) à la place de la simulation actuelle, pour la validation d'inscription et les alertes.
- Génération de rapports PDF en plus du CSV, pour des restitutions plus présentables (utilisation de WeasyPrint ou ReportLab côté Django).

- Mise en production sur un hébergeur (Render, Railway, OVH) avec base de données PostgreSQL et certificat SSL.
- Tests automatisés : tests unitaires Django (pytest-django) sur les modèles et les vues, tests end-to-end avec Cypress sur les parcours utilisateurs critiques.
- Internationalisation : permettre à l'utilisateur de basculer en anglais via une bibliothèque comme i18next.
- Ajout d'une vraie connectivité IoT : intégration MQTT pour recevoir des données réelles d'objets connectés (capteurs Raspberry Pi, ESP32) en plus des données simulées par le seed.

Ce projet nous a confortés dans l'idée que le développement web moderne ne se résume pas à empiler des frameworks, mais à articuler des choix techniques cohérents au service d'un usage. Il a aussi mis en lumière l'importance de la communication au sein d'une équipe, de la documentation, et de l'écoute des retours utilisateurs pour faire évoluer un produit.